Lecture 13: Transformer Architecture

Week 5, Lecture 2 - Attention Is All You Need

PSYC 51.07: Models of Language and Communication

Winter 2026

Today's Agenda 📋

1. 🤖 **The Transformer Revolution**: Why it changed everything
2. 🏗️ **Architecture Overview**: Encoder, Decoder, and components
3. 🎯 **Self-Attention**: The core mechanism (Q, K, V)
4. 📝 **Self-Attention Example**: Understanding pronoun resolution
5. 🎭 **Multi-Head Attention**: Learning diverse relationships
6. 🔍 **Three Types of Attention**: Self, Masked, Cross

Goal: Understand the Transformer architecture and self-attention

The Transformer Revolution 🤖

”Attention Is All You Need”

Before Transformers (2017):

- RNNs/LSTMs with attention
- Sequential processing
- Hard to parallelize
- Limited context window
- Slow training

The Transformer Revolution 🤖

"Attention Is All You Need"

Before Transformers (2017):

- RNNs/LSTMs with attention
- Sequential processing
- Hard to parallelize
- Limited context window
- Slow training

After Transformers:

- Pure attention, no recurrence!
- Parallel processing
- Scales to GPUs/TPUs
- Long-range dependencies
- Fast & effective

Key Innovation

Replace sequential RNNs with **self-attention** layers that process all positions in parallel!

Reference: Vaswani et al. (2017) - "Attention Is All You Need"

Why Get Rid of RNNs? 🚫

Limitations of Recurrent Architectures:

1. **Sequential Bottleneck**
   - Must process token t before token $t + 1$
   - Cannot parallelize across sequence
   - Slow on modern GPUs/TPUs
2. **Long-Range Dependencies**
   - Information must flow through many steps
   - Gradient vanishing/exploding problems
   - Hard to learn dependencies 100+ tokens apart
3. **Memory Constraints**
   - Hidden state must remember everything
   - Limited by fixed-size representation
   - Even with attention, RNN is the bottleneck

Transformer Solution: Every token can attend to every other token directly!

Key Components:

- **Multi-Head Self-Attention**: Relate all positions to each other
- **Feed-Forward Networks**: Transform representations
- **Residual Connections & Layer Norm**: Training stability (not shown)
- **Positional Encoding**: Inject position information

Self-Attention: The Core Mechanism 🎯

Key idea: Each word attends to all other words in the sequence

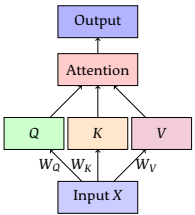
Three learned projections:

- **Query (Q)**: What am I looking for?
- **Key (K)**: What do I contain?
- **Value (V)**: What information do I have?

Computation:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Understanding Query, Key, Value 🔑

Analogy: Database lookup or information retrieval

Information Retrieval:

- **Query**: Your search query
- **Key**: Document titles/keywords
- **Value**: Document contents

Process:

1. Compare query to all keys
2. Get similarity scores
3. Weight values by scores
4. Return weighted combination

Self-Attention:

- **Query**: What token i is looking for
- **Key**: What token j offers
- **Value**: Information from token j

Process:

1. Compare Q_i to all K_j
2. Get attention scores
3. Weight all V_j by scores
4. Return new representation for i

Key Insight

Each token simultaneously acts as:

- A query (what it needs from other tokens)
- A key (how it should be retrieved)
- A value (what information it provides)

Scaled Dot-Product Attention

Step-by-step computation:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

1. **Compute scores**: $S = QK^T$
   - Matrix multiplication: $[n \times d_k] \times [d_k \times n] = [n \times n]$
   - Each element S_{ij} = similarity between token i and j
2. **Scale**: $S' = S/\sqrt{d_k}$
   - Prevents gradients from becoming too small
   - When d_k is large, dot products grow large
   - Scaling keeps softmax gradients stable
3. **Softmax**: $A = \text{softmax}(S')$
   - Convert to probability distribution (rows sum to 1)
   - Each row = attention distribution for one token
4. **Weighted sum**: Output $= AV$
   - $[n \times n] \times [n \times d_v] = [n \times d_v]$
   - Each output is a weighted combination of all values

Self-Attention Example

Sentence: "The animal didn't cross the street because it was too tired"

Question: What does "it" refer to?

Self-attention allows the model to:

- Resolve pronouns
- Understand long-range dependencies
- Capture syntactic and semantic relationships
- Do this in parallel for all positions!

Visualizing the Attention Matrix 🔍

For sentence: "The cat sat on the mat"

To → **From ↓**	The	cat	sat	on	the	mat
The	0.3	0.4	0.1	0.1	0.05	0.05
cat	0.1	orange!800.5	0.2	0.1	0.05	0.05
sat	0.05	0.3	orange!800.4	0.15	0.05	0.05
on	0.05	0.1	0.2	0.3	0.1	0.25
the	0.05	0.05	0.05	0.1	0.3	0.45
mat	0.05	0.05	0.1	0.2	0.2	orange!800.4

Observations:

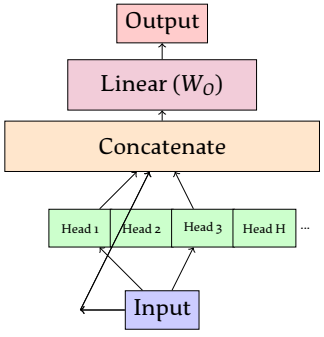
- Each row sums to 1.0 (probability distribution)
- Diagonal elements often high (self-attention)
- "cat" attends to itself and "The" (determiner-noun relationship)
- "mat" attends to "the" (determiner) and "on" (preposition)

Multi-Head Attention 🎭

Why use multiple attention heads?

Intuition:

- Different heads learn different relationships
- Head 1: Syntactic relationships
- Head 2: Semantic relationships
- Head 3: Positional patterns
- Etc.



Formula:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(h_1, ..., h_H)W_O$$
$$h_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Typical: 8-16 heads in practice

BERT-base: 12 heads, BERT-large: 16 heads, GPT-3: 96 heads!

Why Multiple Heads? 🤔

Example: Different heads learn different patterns

Sentence: "The cat sat on the mat"

Head 1: Syntactic Dependencies

- "cat" → "The" (noun-determiner)
- "sat" → "cat" (verb-subject)
- "mat" → "the" (noun-determiner)
- Learns grammar structure

Head 2: Semantic Relations

- "sat" → "mat" (action-location)
- "cat" → "mat" (agent-location)
- Learns meaning relationships

Head 3: Local Context

- Each word → neighbors
- Short-range dependencies
- N-gram like patterns

Head 4: Long-Range

- Distant word relationships
- Document-level context
- Coreference resolution

Ensemble Effect

Multiple heads provide a richer, more diverse representation by attending to different aspects of the input simultaneously!

Three Types of Attention 🔍

1. **Self-Attention (Encoder)**
   - Each position attends to all positions in same sequence
   - Bidirectional: can see past and future
   - Used in: BERT, encoder-only models
2. **Masked Self-Attention (Decoder)**
   - Each position attends only to previous positions
   - Prevents "looking into the future"
   - Used in: GPT, decoder-only models
3. **Cross-Attention (Encoder-Decoder)**
   - Decoder attends to encoder outputs
   - Queries from decoder, Keys/Values from encoder
   - Used in: T5, BART, machine translation

Type	Q from	K, V from
Self-Attention	Sequence	Same sequence
Masked Self-Attention	Sequence	Same sequence (past only)
Cross-Attention	Decoder	Encoder

Masked Self-Attention 🎭

Preventing the model from "cheating" during generation

Problem: During training, we have the full target sequence. Without masking, the model could "peek" at future tokens!

Solution: Mask out future positions by setting attention scores to $-\infty$ before softmax.

Attention scores before masking:

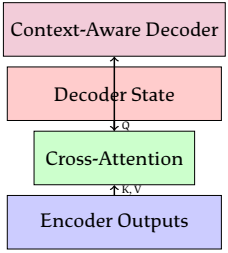
	t_1	t_2	t_3	t_4
t_1	0.5	red!30-inf	red!30-inf	red!30-inf
t_2	0.3	0.4	red!30-inf	red!30-inf
t_3	0.2	0.3	0.4	red!30-inf
t_4	0.15	0.25	0.3	0.35

After softmax: Future positions have weight 0

Result: Token at position t can only attend to positions $\leq t$

Cross-Attention 🔗

Connecting encoder and decoder in seq2seq models



Key Properties:

- **Q** comes from decoder (what decoder needs)
- **K, V** come from encoder (what input provides)
- Allows decoder to ”look at” relevant parts of input
- Similar to the original attention mechanism from Lecture 12!

Used in: Machine translation, summarization, any encoder-decoder task

Implementing Self-Attention in PyTorch 💻

Scaled dot-product attention

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

class SelfAttention(nn.Module):
    def __init__(self, embed_dim):
        super().__init__()
        self.embed_dim = embed_dim
        self.W_q = nn.Linear(embed_dim, embed_dim)
        self.W_k = nn.Linear(embed_dim, embed_dim)
        self.W_v = nn.Linear(embed_dim, embed_dim)

    def forward(self, x, mask=None):
        # x: [batch, seq_len, embed_dim]
        Q = self.W_q(x)  # [batch, seq_len, embed_dim]
```

Computational Complexity

Understanding the cost of self-attention

Operation	Time Complexity	Space Complexity
RNN/LSTM	$O(n \cdot d^2)$	$O(d)$
Self-Attention	$O(n^2 \cdot d)$	$O(n^2)$
Feed-Forward	$O(n \cdot d^2)$	$O(d)$

where n = sequence length, d = embedding dimension

Trade-offs:

Pros of Self-Attention:

- Parallelizable (all positions at once)
- Direct connections between all tokens
- No vanishing gradients

Cons of Self-Attention:

- Quadratic in sequence length
- Memory intensive for long sequences
- Limits context window size

Typical limits:

- BERT: 512 tokens
- GPT-2: 1024 tokens
- GPT-3: 2048 tokens
- Modern models: 4096-100k tokens (with optimizations)

Discussion Questions 💭

1. **Self-Attention vs RNN Attention:**
   - What's the key difference?
   - Why is self-attention more powerful?
   - When might RNNs still be useful?
2. **Query, Key, Value Framework:**
   - Why three separate projections instead of one?
   - What if we used $Q = K = V = X$?
   - How does this relate to information retrieval?
3. **Multi-Head Attention:**
   - Why not just use one big attention head?
   - How many heads is optimal?
   - Can we interpret what each head learns?
4. **Scalability:**
   - $O(n^2)$ is problematic for long documents. Solutions?
   - Sparse attention? Local attention? Other ideas?

Looking Ahead 🔮

What's Next?

Today we learned:

- Why transformers replaced RNNs
- Self-attention mechanism (Q, K, V)
- Multi-head attention
- Three types of attention (self, masked, cross)

Next lecture (Lecture 14 - Training Transformers):

- Positional Encoding: How to inject position information
- Feed-Forward Networks: The other key component
- Layer Normalization & Residuals: Training stability
- FlashAttention: Making transformers faster
- Three Architectures: Encoder, Decoder, Encoder-Decoder
- Practical Tips: Training and using transformers

We're building up to BERT and GPT! 🚀

Summary 🎯

Key Takeaways:

1. **Transformer Revolution**
   - Pure attention, no recurrence
   - Parallel processing, faster training
2. **Self-Attention Mechanism**
   - Query, Key, Value framework
   - Each token attends to all others
   - Scaled dot-product: $\text{softmax}(QK^T/\sqrt{d_k})V$
3. **Multi-Head Attention**
   - Multiple heads learn diverse relationships
   - Concatenate and project back
   - Richer representations
4. **Three Attention Types**
   - Self (encoder), Masked (decoder), Cross (encoder-decoder)
   - Different uses for different architectures

Self-attention is the foundation of modern NLP!

References

Essential Papers:

- **Vaswani et al. (2017) -** "Attention Is All You Need"
  - The original Transformer paper
  - Introduced self-attention, multi-head attention
  - Foundation of modern NLP
- **Bahdanau et al. (2015) -** "Neural Machine Translation by Jointly Learning to Align and Translate"
  - Original attention mechanism (for comparison)

Tutorials and Resources:

- **The Illustrated Transformer** by Jay Alammar
  - https://jalammar.github.io/illustrated-transformer/
  - Visual step-by-step explanation
- **Annotated Transformer** by Harvard NLP
  - https://nlp.seas.harvard.edu/annotated-transformer/
  - Line-by-line implementation
- **HuggingFace Course -** Chapter 1.4
  - How Transformers work

Questions? 🙋

Discussion Time

Topics for discussion:

- Self-attention mechanism
- Query, Key, Value intuition
- Multi-head attention
- Masked vs. unmasked attention
- Implementation questions

Thank you! 🙏

Next: Training Transformers!